1. Práctica 11: Sistema Operativo en Tiempo Real (RTOS)

1.1. Objetivos

- Integrar un sistema operativo en tiempo real en un procesador con capacidad expropiativa.
- Entender cómo se interconexionan los diferentes módulos del RTOS para lograr su funcionalidad.

1.2. Fundamentos teóricos

Un **Sistema Operativo en Tiempo Real** o **RTOS** (*Real-Time Operating System*) es un software diseñado para administrar los recursos del microprocesador y garantizar que las diferentes tareas que pueda realizar, se ejecuten en tiempos predecibles.

En un entorno *bare-metal* estándar, la temporización depende de retardos con espera activa o interrupciones de temporizadores, pero al no existir un supervisor, existe mayor posibilidad de fallos, o tareas que se atascan y bloquean al resto, al contar habitualmente con multitarea cooperativa y no expropiativa.

Un RTOS añade una capa superior de gestión, donde se divide el tiempo en trozos (*time-slicing*) y se va dando a cada tarea según lo necesite, siendo capaz, mediante configuración de prioridades, de ejecutar ciertas tareas en momentos concretos (por ejemplo, para sistemas temporizados críticos como vehículos o maquinaria industrial).

1.2.1. Arquitectura y Planificación Expropiativa en Cortex-M

En esta práctica se utiliza un RTOS de tipo expropiativo sobre la arquitectura ARM Cortex-M7 de nuestro **STM32F723IE**. El kernel puede suspender la ejecución de una tarea en curso y ceder el procesador a otra tarea de mayor prioridad o a la que le corresponda el siguiente turno de ejecución en caso de empate de prioridades.

La interacción entre el hardware del microprocesador y el núcleo del RTOS se realiza mediante tres interrupciones del Cortex-M:

- **SysTick** Generará las interrupciones periódicas que indicarán el cambio de contexto entre las diferentes tareas en ejecución del sistema.
- **SVC (Supervisor Call):** Es una excepción provocada por software mediante la instrucción assembly `svc 0`. Se utiliza durante la fase de arranque para realizar la transición inicial del modo privilegiado del procesador al modo de ejecución de la primera tarea del RTOS (`vTaskStartScheduler`).
- **PendSV (Pendable Service Call):** Es una excepción que se configura con la prioridad de hardware **más baja posible** en el NVIC. La función de `PendSV` es ejecutar el cambio de contexto (*context switch*). Al tener la menor prioridad, se garantiza que cualquier interrupción de periféricos (`UART` , `GPIO` , `TIMx`) finalice antes de que el RTOS intervenga.

1.2.2. Bloque de Control de Tarea (TCB) y Estructuras Internas

Para gestionar la ejecución multitarea, el kernel del RTOS asocia a cada tarea una estructura llamada **TCB** (*Task Control Block*). El TCB contiene los datos necesarios sobre la tarea como para que el RTOS sea capaz de retirarla de ejecución o reanudarla cuando el planificador así lo dictamine.

- Puntero a la Cima de Pila (`pxTopOfStack`): Apunta a la posición actual del puntero de pila de la tarea. Al ejecutarse un cambio de contexto, los registros de la CPU ($R0$-$R12$, LR, PC, $xPSR$) se guardan en esta pila y se actualiza este puntero. Así se preserva el estado actual
- Punteros a las listas internas del kernel bloqueadas o suspendidas para vincular directamente la tarea a una de ellas.
- Prioridad de la Tarea (`uxPriority`): Determina el nivel de preferencia de ejecución asignado a la tarea. Cuanta más prioridad tenga, más veces entrará en ejecución, aunque en última instancia esto depende de la política de planificación.
- Límite de la Pila (`pxStack`): Puntero a la base de la pila utilizada opcionalmente para detectar desbordamientos (*stack overflow*).

El kernel mantiene internamente varias listas enlazadas ordenadas por prioridad. Cuando la interrupción periódica `xPortSysTickHandler` (`Systick_Handler`) se ejecuta, el planificador consulta la lista de tareas bloqueadas y la lista de tareas listas. La función `vTaskSwitchContext` simplemente localiza el TCB de mayor prioridad disponible y actualiza el puntero `pxCurrentTCB`.

1.2.3. Gestión de Memoria: Pila (Stack) y Memoria Dinámica (Heap)

En un sistema *bare-metal* existe un único espacio de pila (*Main Stack Pointer* o **MSP**) utilizado por todo el código y por las interrupciones. En un RTOS, con la introducción de las tareas, la estructura se complica como puede verse en la Figura 1.

- **MSP (Main Stack Pointer):** Se utiliza durante la fase de arranque, y la gestión del RTOS (planificador y cambio de contexto), así como para atender la ejecución de los manejadores de interrupción (ISRs). Esto último es especialmente interesante pues así no es necesario que cada tarea reserve espacio en la pila para todas las posibles interrupciones que sufra.
- **PSP (Process Stack Pointer):** Cada tarea creada dispone de su propio espacio de pila independiente en la memoria RAM, al que el hardware conmuta automáticamente cuando la tarea entra en estado *Running*.
- **Gestión de Heap:** FreeRTOS reserva una región contigua en la sección `.bss` de la RAM. Al crear una tarea, el kernel asigna automáticamente en este heap la memoria necesaria tanto para el TCB como para la pila privada de la tarea.

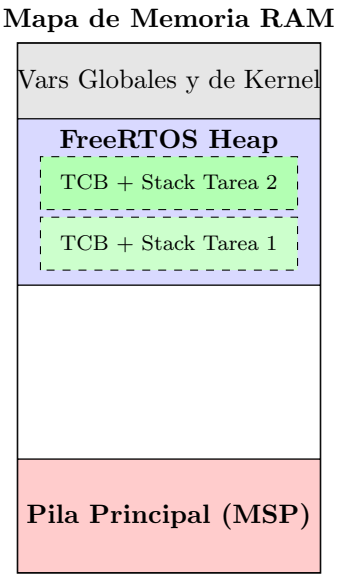


Figura 1: Disposición de la memoria RAM: separación entre el MSP (utilizado por las interrupciones) y las pilas independientes de cada tarea (PSP) alojadas en el Heap de FreeRTOS.

1.2.4. Diagrama de Interacción entre Interrupciones y Tareas

La Figura 2 ilustra el flujo temporal de interacción entre el temporizador de sistema, las tareas de usuario y el mecanismo de cambio de contexto de FreeRTOS.

1.3. Desarrollo de la práctica

El primer paso es descargarse el paquete completo de FreeRTOS. Para ello descargamos el siguiente repositorio:

```
> git clone https://github.com/FreeRTOS/FreeRTOS-LTS/
```

Y nos aseguramos de estar en la rama `202604-LTS` (es con la que se han hecho las pruebas, si hay una más nueva puedes probar también, pero quizá cambien detalles de la configuración).

```
> cd FreeRTOS-LTS
> git checkout 202604-LTS
```

El repositorio es bastante extenso, pero a nosotros nos interesan unos archivos muy concretos, marcados en rojo en el siguiente diagrama. Echa un vistazo en el interior para mirar, a vista de pájaro, los contenidos de cada uno. Normalmente, la selección de qué ficheros se utilizan se hace desde archivos de configuración globales, pero nosotros lo vamos a hacer a mano por simplicidad.

```
FreeRTOS-Kernel/ ..... – Código fuente de las prácticas
├── list.c ..... – Código fuente de lista enlazada
├── queue.c ..... – Código fuente de cola
├── tasks.c ..... – Código fuente de gestión de tareas
├── timers.c ..... – Código fuente de gestión de timers
└── examples/ ..... – Ejemplo de uso de FreeRTOS
    └── cmake_example/ ..... – Ejemplo de aplicación
        └── main.c/ ..... – Archivo de aplicación
```

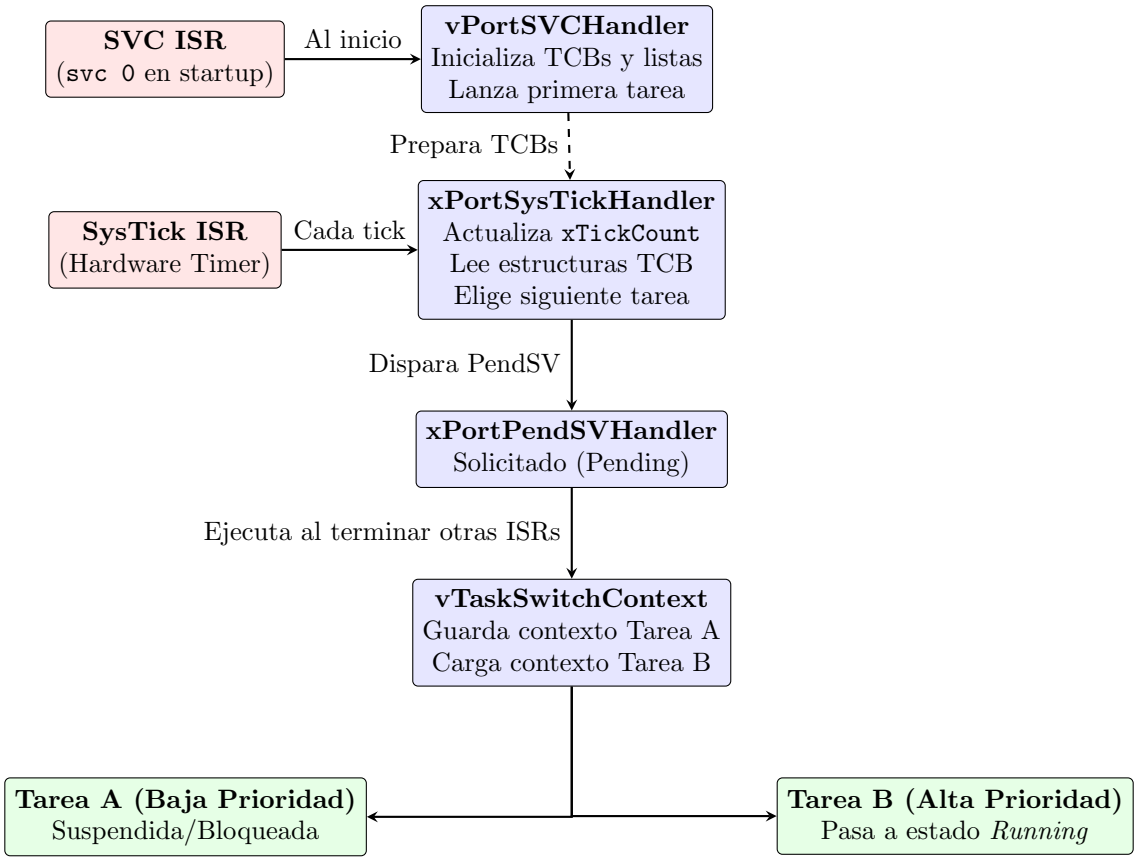


Figura 2: Flujo completo de ejecución: arranque mediante excepción SVC, inspección de estructuras TCB en SysTick y cambio de contexto diferido con PendSV.

```
│   │
│   ├── template_configuration/ ..... – Ejemplo de configuración
│   │   └── FreeRTOSConfig.h/ ..... – Archivo de configuración
├── include/ ..... – Cabeceras de los diferentes componentes de FreeRTOS
│   ├── deprecated_definitions.h ..... – Definiciones en desuso
│   ├── FreeRTOS.h ..... – Definiciones de macros, funciones y estructuras utilizadas
│   ├── list.h ..... – Cabecera de lista enlazada
│   ├── mpu_wrappers.h ..... – Cabecera utlizada con unidad de protección de memoria
│   ├── portable.h ..... – Cabecera para incluir los ports de procesador específicos
│   ├── projdefs.h ..... – Definiciones genéricas de tipos y errores
│   ├── queue.h ..... – Cabecera de cola
│   ├── stack_macros.h ..... – Cabecera para controlar el stack overflow opcionalmente
│   ├── task.h ..... – Cabecera para gestión de tareas
│   ├── timers.h ..... – Cabecera para gestión de timers
├── portable/ ..... – Código específico de los diferentes procesadores soportados
│   ├── GCC/ ..... – Código para compiladores GCC
│   │   └── ARM_CM7/ ..... – Código para procesadores ARM-M7
│   │       └── r0p1/ ..... – Versión r0p1 del procesador
│   │           ├── port.c ..... – Funciones específicas de procesador
│   │           └── portmacro.h ..... – Macros y definiciones específicas de procesador
│   ├── MemMang/ ..... – Código para gestión de memoria
│   │   └── heap_4.c ..... – Código fuente para gestión de memoria dinámica
```

Ahora debemos configurar todos los elementos de la práctica. En concreto, configurar FreeRTOS, hacer el puente con nuestro código y BSP, y finalmente integrarlo todo en un código main.

El primer paso es editar el archivo `FreeRTOSConfig.h` para poner la configuración concreta de nuestro procesador. Debemos encontrar las siguientes definiciones (algunas son nuevas) y añadirlas o editarlas en el archivo:

```
//1)  Para que la temporización funcione bien, debemos indicar la frecuencia
//    de nuestro reloj (16Mhz), y el tick rate que queremos (en este caso)
//    lo configuramos para 1 tick cada milisegundo
#define configCPU_CLOCK_HZ    ( ( unsigned long ) 16000000 )
#define configTICK_RATE_HZ                          1000

//2)  Ahora configuramos las prioridades para las interrupciones del kernel
//    (para la planificación). No configurar esto hace que la compilación no
//    funcione
#define configPRIO_BITS                                 4
#define configLIBRARY_LOWEST_INTERRUPT_PRIORITY       15
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY  5

#define configKERNEL_INTERRUPT_PRIORITY \
    ( configLIBRARY_LOWEST_INTERRUPT_PRIORITY << (8 - configPRIO_BITS) )

#define configMAX_SYSCALL_INTERRUPT_PRIORITY \
    ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 - configPRIO_BITS) )

//3)  Quitamos las comprobaciones de stack overflow pues requiere de
//    alguna implementación adicional
    #define configCHECK_FOR_STACK_OVERFLOW        0
```

Lo siguiente será puentear las interrupciones de sistema a las definidas en `port.c`. Son tres:

1. `vPortSVCHandler`: Gestiona la excepción de llamada a supervisor (en este caso el supervisor es el RTOS). Hay que hacer que se llame ante la interrupción `SVC` mapeando el

manejador por defecto `SVC_Handler`.

2. `xPortPendSVHandler`: Gestiona la excepción de llamada pendiente para el cambio de contexto. Este tipo de interrupción tiene la menor prioridad posible, para que todas las otras interrupciones (periféricos principalmente) tengan prioridad sobre ella. Se guarda el contexto de la tarea actual en el stack, y se llama al planificador para cambiar de tarea. Hay que hacer que se llame ante la interrupción `PendSV` mapeando el manejador por defecto `PendSV_Handler`.

3. `xPortSysTickHandler`: Gestiona las interrupciones del ticker del sistema para que el RTOS lleve la cuenta del tiempo y pueda planificar las tareas de manera acorde. Hay que hacer que se llame ante la interrupción `SysTick` mapeando el manejador por defecto `SysTick_Handler`.

Esta configuración se debe hacer desde el archivo de inicialización `startup_stm32f723iekx.s` con las siguientes instrucciones:

```
.weak   SVC_Handler
.thumb_set SVC_Handler,vPortSVCHandler

.weak   PendSV_Handler
.thumb_set PendSV_Handler,xPortPendSVHandler

.weak   SysTick_Handler
.thumb_set SysTick_Handler,xPortSysTickHandler
```

Ahora solo quedará gestionar el archivo principal `main.c`, que tendrá esta forma:

```c
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

#include "bsp.h"
#include "stdio.h"

void Task1(void *pvParameters) {
    for (;;) {
        //... completar hacer toggle a un LED (por ejemplo PA5)
        vTaskDelay(pdMS_TO_TICKS(500)); // Delay 500 ms
    }
}


void GPIO_Init(void) {
    //... completar inicializar los GPIOs que se vayan a utilizar
}

int main(void) {
    // 1. Initialize peripherals
    GPIO_Init();


    // 2. Create FreeRTOS Tasks
    xTaskCreate(Task1, "LED_Task1", 128, NULL, 1, NULL);
    //... completar crear más tareas que enciendan por ejemplo otros leds

    // 3. Start Scheduler (never returns)
    vTaskStartScheduler();

    while (1); // aquí no debería llegar nunca
```

```
33 }
```

Es importante saber que no podemos (o no debemos, mejor dicho) inicializar manualmente componentes del sistema como `SysTick`, ya que ahora lo maneja nuestro RTOS. También hay que tener en cuenta que se modifcarán parámetros del `NVIC` automáticamente, por lo que hay que tener especial cuidado al añadir nuevas interrupciones, sin enmascarar otras.

1.4. Para hacer más

Los sistemas operativos en tiempo real ofrecen funciones para sincronización de tareas mediante semáforos, mutex, variables condicionales y demás mecanismos que, mediante la atomicidad, aseguran ciertos comportamientos ante condiciones de carrera. Investiga cómo se resuelve esto en RTOS, y qué opciones hay al respecto.